

Supplemental Material for “Container Overhead or Stale Dependencies? Decomposing the Latency and Energy Cost of Containerised Inference on an Edge Node”

Lin Ding Shan
UCSI University, Kuala Lumpur, Malaysia

This document supports the main paper. Section numbers, equations and tables cited as “paper” refer to the main text. Every table here is generated from the raw run logs by `analysis/make_supp.py` in the released repository; nothing is typed by hand.

S1 Run schedule, integrity checks and deviations

Table S1 lists the nine runs in block order, with the within-block order as randomised (seed 20260927). Before each run, the orchestrating script (`orchestrate.py`) (i) waited until the battery charge current was at most +50 mA (“floating”); (ii) waited until the SoC temperature was at most 50 °C; (iii) from `legacy_2` onward, disabled the camera’s dynamic frame rate on the host with `v4l2-ctl` and saved the full camera control state next to the run; (iv) launched the run; and afterwards (v) checked that `RUN_INFO.txt` reported completion and that the frame count lay within 0.93–1.05 of 129,600. A run failing any check stopped the orchestrator.

Camera frame rate. The USB camera runs in aperture-priority auto exposure. With `exposure_dynamic_framerate` enabled, it lowers its frame rate when the scene is dark. `legacy_1` (03:56–06:56) and `matched_1` (06:57–09:57) ran in a dark room; the median per-second rate was 15.0 frame/s in every hour of both runs, and the overall rates were 15.046 and 15.049 frame/s. The original `native_1` run began at 09:58 (`orchestrator.log`); a check of its partial log during the run showed the frame rate rising towards 30 frame/s after the room lights came on; the run was stopped, the orchestrator removed its directory before the replacement run, so this observation cannot be checked from the deposited data. All later runs, starting with `legacy_2` and including the replacement `native_1`, requested 15 frame/s with dynamic frame rate disabled, and ran at 14.88–14.91 frame/s. The loop is camera-bound in all nine runs, so the rate fixes the number of inferences per active phase.

Harness update and image rebuild. The harness used for `legacy_1` and `matched_1` did not request a frame rate and did not write the “Measured rate” line to `RUN_INFO.txt`. It was updated before `legacy_2` to add the `-fps` option and that line; the preprocessing, the timed `InferenceSession.run` call and the per-frame logging were not changed. The earlier version of the harness was not retained. The images copy the harness in their last layer, after the layers that install the stack, and were rebuilt to include it; unless the build cache is bypassed, such a rebuild reuses the stack layers. Image layer digests were not recorded. The environment fingerprints (Table S4) were taken on 27 September at 02:03, before the first run, and were not repeated after the rebuild; the per-run `provenance.json` records show the same interpreter, glibc, ONNX Runtime, NumPy and OpenCV versions in all runs of each condition.

Replicate field. The “Replicate” line of each `RUN_INFO.txt` holds the run’s position in the nine-run schedule (1–9), not its replicate number within the condition; the directory label (e.g. `native_2`) identifies the replicate.

Block 1. The replacement `native_1` ran from 20:19 to 23:19 on the same day, after block 2 and immediately before block 3. Block 1 is therefore not a contiguous session. The paper’s Section 4.5 repeats the analysis on blocks 2 and 3 only.

Host state. All systemd timers (package updates, manual-page indexing, swap write-back and others) were stopped for the whole campaign; the host software was not changed between fingerprinting and the last run. The graphical desktop session (`labwc` compositor, panel and file manager) remained running in every run and used under 1% CPU. Provenance records show the same kernel (6.18.39+rpt-rpi-2712), governor (`ondemand`), four-CPU affinity and model SHA-256 (bd79f604...eb7ccb752) in all nine runs; container runs recorded `cgroup cpu.max = max 100000` (no quota). The firmware throttle word, readable only on the host, was 0x0

before each native run; in all runs the 1 Hz sysfs sampler, identical in all conditions, recorded no frequency cap and no under-voltage alarm (Table S1).

Table S1: Run schedule and integrity. Start times are local (UTC+8). Frames: all inferences in the run; analysed: after the 600 s warm-up. Battery current: mean (min, max) over the whole run, positive into the pack. Cap/UV: 1 Hz samples with the CPU frequency capped below 2400 MHz / under-voltage alarm set.

Block	Run	Condition	Start	Frames	Analysed	Rate (frame/s)	Active / sleep (s)	Battery I (mA)	Bus V (V)	Cap / UV
1	legacy_1	legacy	2026-09-27 03:56:12	130,093	122,878	15.046	60.05 / 15.02	5.0 (-12, 39)	15.306	0 / 0
1	matched_1	matched	2026-09-27 06:57:14	130,054	122,843	15.049	60.02 / 15.00	1.8 (-24, 27)	15.296	0 / 0
1	native_1	native	2026-09-27 20:19:29	128,922	121,775	14.910	60.06 / 15.00	0.9 (-27, 25)	15.293	0 / 0
2	legacy_2	legacy	2026-09-27 11:15:22	128,865	121,720	14.906	60.04 / 15.02	1.1 (-21, 23)	15.292	0 / 0
2	native_2	native	2026-09-27 14:16:24	128,923	121,776	14.910	60.06 / 15.00	1.0 (-18, 25)	15.298	0 / 0
2	matched_2	matched	2026-09-27 17:17:27	128,895	121,748	14.907	60.05 / 15.00	1.2 (-17, 26)	15.298	0 / 0
3	matched_3	matched	2026-09-27 23:22:31	128,788	121,639	14.896	60.04 / 15.00	0.7 (-27, 24)	15.298	0 / 0
3	legacy_3	legacy	2026-09-28 02:23:34	128,573	121,444	14.879	60.02 / 15.02	0.9 (-21, 24)	15.298	0 / 0
3	native_3	native	2026-09-28 05:24:36	128,577	121,448	14.877	60.03 / 15.00	1.0 (-20, 24)	15.292	0 / 0

S2 Per-run results

Table S2 gives every run-level quantity used in the paper, and Table S3 gives the within-block differences from which each effect and interval in the paper’s Table 2 and Fig. 3 is computed. The effect for a quantity is the mean of the three block differences d_1, d_2, d_3 , and its 95% interval is $\bar{d} \pm t_{0.975,2} s_d / \sqrt{3}$.

S3 Environment fingerprints and image definitions

Table S4 gives the fingerprints. The matched image definition (`Dockerfile.matched`) was generated by `make_matched_dockerfile.py` from the running host interpreter; it installs Debian’s `python3` package in `debian:trixie-slim` and pins every package to the host’s version. Fingerprints were taken with `fingerprint.py` on the host and inside both images, and the host and matched records compare equal as whole JSON documents.

Legacy image. The first campaign’s image could not be recovered, so condition C was rebuilt from its definition (`Dockerfile.legacy`). The original definition installed `onnxruntime` without a version constraint on `python:3.9-slim-bullseye`. On 64-bit ARM with CPython 3.9, the last ONNX Runtime release that publishes a compatible wheel is 1.19.2, so that version is determined. The other packages are pinned to the versions in the paper’s Table 1; the first campaign’s exact resolution of those packages cannot be established. The original definition also installed the system package `libglib2.0-0`, which the base distribution’s archive no longer serves; it was omitted. The headless OpenCV build does not load it, and the harness imports and runs unchanged. The ONNX Runtime build strings (Table S4) show that the host and legacy runtimes are different builds, with the legacy build recording its compiler flags.

S4 Instrumentation details

Per-frame log (`basil_data.csv`): wall-clock timestamp; latency of the `InferenceSession.run` call, measured with `time.time()` and logged at 0.1 ms resolution; instantaneous frame rate; system-wide CPU utilisation since the previous frame (`psutil`); memory utilisation; SoC temperature; throttle flag; predicted class and confidence; the most recent 1 Hz supply reading.

1 Hz log (`power_log.csv`), written through both phases: phase; Type-C bus voltage, current and power (UPS HAT (E) registers 0x10, 0x12, 0x14); battery voltage, signed current and charge (0x20, 0x22, 0x24); SoC temperature; CPU utilisation over the previous second; current CPU frequency and frequency cap from `cpufreq`; the `rpi_volt` low-voltage alarm; fan speed and PWM from `hwmon`. The sampler runs as a thread in the measured process and reads the same sysfs files in all three conditions.

Cycle log (`cycle_events.csv`) marks each active and sleep transition. Phase durations in Table S1 are computed from it.

Consistency check. Run means of bus current and of bus power divided by bus voltage agreed within 0.4% in every run. Individual samples disagree by more, because the module’s registers are not updated together. The vendor does not specify the module’s absolute accuracy.

Table S2: Per-run results after warm-up. Latency in ms; CPU ms: CPU time per inference; P_{floor} : settled sleep-phase floor; energies per inference in mJ, Eqs. (1)–(3) of the paper; temperatures in °C (active-phase mean, mean cyclic peak, maximum); fan and frequency are active-phase means.

Run	Lat	mean	median	p95	p99	SD	CPU %	RAM %	P_{act}	W	P_{slp}	W	P_{floor}	W	E_{base}	E_{inc}	E_{cyc}	J/cycle	T act.	T peak	T max	Fan rpm	MHz
native_1	19.51	20.30	26.30	28.90	3.85	56.24	8.59	10.541	7.020	6.674	559.5	265.2	824.6	738.4	64.27	66.54	68.3	5550	2399				
native_2	19.24	20.00	26.40	29.70	4.01	55.93	8.58	10.453	6.972	6.617	554.7	263.3	817.9	732.4	61.45	63.68	65.5	5179	2399				
native_3	19.38	20.30	25.30	28.10	3.77	56.07	8.60	10.381	6.911	6.554	550.7	263.2	813.9	726.8	62.14	64.43	65.5	5278	2399				
matched_1	21.15	22.50	25.00	33.20	3.97	58.12	8.79	10.548	6.974	6.650	552.4	264.4	816.8	737.8	62.52	64.68	66.1	5309	2399				
matched_2	20.50	20.00	26.80	30.50	4.15	57.01	8.79	10.500	6.929	6.564	550.4	270.1	820.5	734.5	61.95	64.22	67.2	5223	2398				
matched_3	20.39	20.00	26.80	30.66	4.21	56.90	8.82	10.496	6.953	6.579	552.0	269.3	821.3	734.6	61.76	64.06	68.3	5171	2398				
legacy_1	28.60	25.30	43.60	44.00	5.44	48.55	8.71	9.713	6.735	6.549	544.1	213.3	757.5	684.4	58.16	60.27	62.2	3907	2393				
legacy_2	30.18	29.70	41.40	43.70	4.90	49.03	8.71	10.030	7.036	6.708	562.6	228.3	790.9	707.9	61.72	63.82	65.5	5302	2399				
legacy_3	29.01	26.30	39.70	43.80	4.90	48.16	8.72	9.702	6.668	6.348	533.4	230.8	764.2	682.4	58.74	60.75	62.2	4354	2398				

Table S3: Within-block differences. Each effect in the paper is the mean of the three block differences shown; the 95% interval uses $t_{0.975,2} = 4.303$.

Quantity	B – A				C – B				C – A			
	blk 1	blk 2	blk 3	mean	blk 1	blk 2	blk 3	mean	blk 1	blk 2	blk 3	mean
Latency, mean (ms)	1.637	1.262	1.013	1.304	7.449	9.678	8.619	8.582	9.086	10.940	9.632	9.886
Latency, p95 (ms)	-1.300	0.400	1.500	0.200	18.600	14.600	12.900	15.367	17.300	15.000	14.400	15.567
Latency, p99 (ms)	4.300	0.800	2.562	2.554	10.800	13.200	13.138	12.379	15.100	14.000	15.700	14.933
CPU utilisation (pp)	1.882	1.072	0.827	1.260	-9.577	-7.972	-8.741	-8.763	-7.694	-6.900	-7.914	-7.503
Power, active (W)	0.007	0.047	0.115	0.056	-0.835	-0.470	-0.794	-0.700	-0.828	-0.423	-0.679	-0.643
Power, sleep (W)	-0.046	-0.043	0.042	-0.016	-0.239	0.107	-0.284	-0.139	-0.284	0.064	-0.242	-0.154
Energy/inf., above floor (mJ)	-0.75	6.84	6.03	4.04	-51.07	-41.77	-38.44	-43.76	-51.82	-34.93	-32.42	-39.72
CPU time/inference (ms)	3.609	2.901	2.031	2.847	-25.433	-21.378	-23.325	-23.379	-21.824	-18.477	-21.295	-20.532
Energy/inf., cycle (mJ)	-7.85	2.56	7.37	0.69	-59.30	-29.56	-57.05	-48.64	-67.15	-27.00	-49.69	-47.95
Cyclic peak (°C)	-1.854	0.532	-0.373	-0.565	-4.409	-0.393	-3.308	-2.703	-6.263	0.140	-3.681	-3.268
Fan, active (rpm)	-240.9	43.2	-106.4	-101.4	-1401.4	79.7	-817.8	-713.2	-1642.3	122.9	-924.2	-814.5

Table S4: Environment fingerprints: SHA-256 of the compiled extension module imported by each numerical package, and the ONNX Runtime build string. Host and matched container are identical in every field.

Package	Host = matched container	Legacy container
onnxruntime	1.29.0 5574a1e5d1dc5bcf1dff1e4ac2d41f8a 9cd7db5ad35661c28ba1433897ceb23e	1.19.2 4e3e7d9ac54723d6602cf0258041a9ef 79bb3c40eb28e9c63d4d105a7d1ce5bc
numpy	2.5.2 72b6f54f37d34fdc2901fb770030bc9f c22f5b1e5f8d0eac9745ca5a349fecdb	1.26.4 147aff3b8942204bbd523583c39b7e6c 5ddd20387afc9126e65183c0c53fc129
opencv-python-headless	5.0.0.93 b69eef7f164dd20965ee3d88221d041c 58d8f6a1e18c4c3bc29588286a7d6eee	4.10.0.84 2c06682a72c65e01542095239c3fbfc0 80156803e5c80972ffad2e5f15424e23
psutil	7.2.2 463d351dd4b4d93957d30a7b645dd0bc cd09276fd5d26c0ebd8c10e5088bala3	6.1.0 9e0377b6a81b798b1c7b3a9388f3ef60 7ea9a1508c1ac4ff632c6b1f1e7aa192
smbus2 (pure Python)	0.6.1	0.5.0
Python / glibc	3.13.5 / 2.41	3.9.23 / 2.31

ONNX Runtime build string, host and matched: git-branch=HEAD, git-commit-id=2e2543fbe9, fp8-kv-cache=1, buildtype=Release

ONNX Runtime build string, legacy:

git-branch=HEAD, git-commit-id=ffceed9d44, buildtype=Release, cmakecxxflags:-DNDEBUG-Wp, -D_FORTIFY_SOURCE=2-Wp, -D_GLIBCXX_ASSERTIONS-fstack-protector-strong-O3-pipe-ffunction-sections-fdata-sections-Wno-restrict-DCPUINFO_SUPPORTED

Sleep-phase floor. The `ondemand` governor held the CPU at 2400 MHz for about 5 s after each active phase and then stepped down to 1500 MHz; node power fell with it. The settled floor P_{floor} is the mean of sleep-phase samples taken at least 6 s after the phase began. The energy split in the paper, Eqs. (2)–(3), uses this floor. Table S2 also gives the plain sleep-phase mean P_{slp} .

Warm-up and cycles. The first 600 s of every run, timed from the run’s first cycle event, are excluded; the remaining 136 complete duty cycles per run are analysed. Cyclic peak temperature is the maximum 1 Hz SoC temperature reading within each active phase (and the following 2 s), averaged over the 136 cycles.

S5 Camera settings

The saved control state (`runs/<label>.camera.txt`, the seven runs from `legacy_2` onward) shows the same settings in every run:

- `auto_exposure=3` (aperture priority);
- `exposure_dynamic_framerate=0`;
- `gain=255, power_line_frequency=2` (60 Hz);
- automatic white balance and continuous autofocus enabled;
- 640×480 at a requested 15 frame/s.

The anti-flicker setting is 60 Hz, whereas the local mains frequency is 50 Hz. This may affect image exposure under artificial light, but it was the same in all runs and does not enter the timed inference call.

S6 Reproducing the analysis

From the repository root, with the deposited runs extracted to `runs/`:

```
python3 analysis/analyse_v2.py      # -> out/numbers_v2.json
python3 analysis/make_tex.py        # -> out/numbers_v2.tex, out/table_results.tex
python3 analysis/make_supp.py       # -> out/supp_tables.tex
python3 analysis/figures_v2.py      # -> figures/fig1..fig5 (.pdf, .png)
```

The paper's text, abstract and tables take their numbers from `numbers_v2.tex` and `table_results.tex`, so a rebuild from a fresh analysis cannot silently diverge from the data; the first-campaign values are read from the repository's earlier analysis (`analysis/numbers.json`, produced by `analysis/recompute.py`). Fixed parameters of the analysis (the 600 s warm-up, the 6 s settling time and the 17.5 ms boundary between latency modes) are set in `analyse_v2.py`. `manuscript/build.sh` runs the whole chain and compiles both documents. The run kit (`runner/`) contains the harness, orchestrator, image definitions and fingerprinting script as used in the campaign, and an as-executed description of the run procedure, including the changes made during the campaign (`PROTOCOL.md`).